

Робоскоп: руководство по согласованному процессу

Версия модели workflow-2.1. Дата: 29 сентября 2026 года.

Назначение и границы

Сервис помогает руководителю склада и специалисту по автоматизации сравнить составы исполнителей для заданного процесса. Полный процесс реализован для склада. Базовый модуль по адресу /legacy/ сохраняет выбор склада, аэропорта и медицинского учреждения, их параметры, каталог и гостевую демонстрацию. Подробная модель аэропорта и медицины не заявляется.

Новая модель сохраняет последовательность и параллельные связи, которые задал клиент. Каждая паллета проходит все операции. Ветвление означает обязательные параллельные работы. Разделение потока по долям и альтернативные маршруты в этом прототипе не моделируются.

Результат является предварительной оценкой и требует верификации при обследовании объекта. Договоры, оплата, доставка и установка представлены учебными переходами. Реальных денежных операций и юридического подписания нет.

Запуск и хранение

Python 3.12, Django, PostgreSQL для сервера. SQLite только для локальной демонстрации. Зависимости зафиксированы в requirements.lock. Dockerfile и docker-compose.yml обеспечивают воспроизводимое развёртывание. Render использует render.yaml и scripts/render_start.py. База подключается через DATABASE_URL. Секреты задаются переменными окружения и не входят в исходный код.

После установки зависимостей: `python manage.py migrate`, `python manage.py createsuperuser`, `python manage.py initialize_workflow`, `python manage.py collectstatic --noinput`, `python scripts/dev.py`. Адрес локального приложения: `http://127.0.0.1:8765`. Внешняя публикация требует `DEBUG=0`, корректных `ALLOWED_HOSTS` и `HTTPS`. Подробности в `DEPLOYMENT.md` и `RENDER_FREE.md`.

`Initialize_workflow` добавляет синтетический набор, проверенные неполные справочные карточки из PDF и черновики принятого парсера. Повторный запуск сохраняет существующие версии. Команда не создаёт пароли. На сервере существующий `seed_data` может создать администратора из `ADMIN_USERNAME` и `ADMIN_PASSWORD` при первом запуске.

Бесплатный Render может засыпать. Срок бесплатной PostgreSQL ограничен. Команда проверяет доступность стенда и сохраняет резервную копию перед сдачей. Неделя хранения требует действующего экземпляра базы на весь период.

Пользовательский путь

1. Зарегистрироваться или войти, открыть «Мои проекты», создать склад.

2. Заполнить параметры, операции, связи, проезды и финансовые допущения. Значения по умолчанию являются учебными допущениями команды. Для загрузки доступен CSV-шаблон проекта.
3. Запустить подбор. Сначала показываются полностью подтверждённые варианты, далее варианты с допущениями. В каждой группе порядок определяется минимальным ТСО доступного сценария на выбранном горизонте.
4. Открыть конфигурацию, проверить состав, очереди, предел проверенной нагрузки, расходы и чувствительность. Пользователь может назначить исполнителей вручную. Одинаковое имя группы означает общий ресурс для нескольких операций.
5. Запустить, остановить или перезапустить воспроизведение событий. Скорость изменяется переключателем. PNG сохраняет текущий кадр, JSON сохраняет полный результат.
6. Выгрузить PDF и Excel, выбрать покупку или аренду и направить заявку всем поставщикам конфигурации.
7. Согласовать все части. Альтернатива поставщика проходит проверку всей цепочки и требует новой заявки для выбранного состава.
8. В учебном заказе последовательно отметить договоры всех поставщиков, оплату, доставку и установку каждой части, затем общую приёмку.

Проект можно скопировать и удалить. Удаление включает версии, расчёты, связанные заявки и учебные заказы. Исходный файл параметров после разбора не сохраняется, его данные находятся внутри версий проекта.

Парсер и база данных

Принятый пакет размещён в `vendor/robot_parser`. Парсер работает офлайн и выдаёт JSONL, одна JSON-запись на строку. Он не подключается к PostgreSQL, не знает пароль базы и не публикует карточки.

Путь интеграции: исходные снимки, парсер, проверка `validate_export.py`, Django importer, черновики `EquipmentRevision`, модерация, указатель `Equipment.published`, публичный каталог.

Администратор может выполнить `python manage.py import_robot_parser --user ADMIN` для встроенного набора или указать `--folder` с готовым результатом. В кабинете доступна загрузка ZIP с `cards.jsonl`, `observations.jsonl`, `offers.jsonl`, `issues.jsonl`, `snapshots.jsonl`, `scenario_links.jsonl` и `catalog_rows.jsonl`. Снимки источников должны сохранять пути `stored_path`. Ограничение ZIP: 5 МБ, после распаковки 20 МБ и 1000 файлов. Программы из загруженного архива не выполняются.

Сервер проверяет контракт, ссылки, контрольные суммы и пути. Извлечённые предложения, наблюдения, замечания и исходные строки сохраняются в БД. Неоднозначная цена, максимальная скорость и результаты внедрения всего склада не подменяют тариф, рабочую скорость с грузом и производительность одного робота.

Сопоставление SKU и комплектаций требует явного подтверждения. У поставщика собственный ассортимент и свои заявки. Администратор назначает участников поставщика в `/admin/`. Для обычного ассортимента используется единый CSV из кабинета. Ошибочные строки возвращаются отдельно, корректные создают черновики. Роль администратора

нельзя получить при обычной регистрации.

Каталог и источники

Использованы catalog_export_v4.csv, пять текстовых снимков принятого парсера и выбранные складские карточки PDF «ФЦ БАС: Каталог внедрения 2008 1247», страница 6. Дополнения организатора упоминают catalog_export_v5. В полученном наборе используется v4, сведения не выдаются за v5.

Три синтетические модели полностью заполнены для проверки смешанного парка, аренды и покупки. Их названия и поставщики явно содержат «Демо». Эти цены и ТТХ не являются предложениями реальных производителей. Реальные неполные карточки не участвуют в расчёте при нехватке критических параметров.

Для каждой версии сохраняются источник, дата создания и доказательства. Обновление каталога не переписывает старый расчёт. Пользователь получает уведомление и самостоятельно запускает актуализацию. Оценки по аналогам можно внести вручную с основанием и отметкой. Автоматический поиск и подбор аналогов пока не реализован.

Подбор и симуляция

Проверяются назначение, грузоподъёмность, габариты и зазор, минимальный проход, высота подъёма, температура, скорость с грузом, автономность, зарядка и производительность конкретной операции. Критический пропуск останавливает расчёт. Несовместимость или недостаточная производительность закрывают экономику.

Кандидаты: до трёх моделей на операцию, до 24 комбинаций, отдельные и общие группы. Число активных исполнителей определяется суммарной длительностью задач и зарядки при целевой загрузке по умолчанию 0,8. Резерв составляет 10% с округлением вверх и не увеличивает рабочую производительность. Выводится до восьми вариантов. Глобальный оптимум рынка не заявляется.

SimPy моделирует 120 паллет, ограниченные буферы и места операций, группы исполнителей, зарядные станции и пропускную способность проездов. Проезд занимает ресурс на время операции с маршрутом, что является консервативным допущением. Общий исполнитель получает только своё фактическое рабочее время. Проверяются завершение заданий, рост задержки, фактический поток и аналитическая верхняя граница с учётом зарядки. Предел нагрузки находится дополнительными прогонами, а не назначается произвольным процентом.

Координаты зон и движение значков схематические. Проверка точных траекторий, радиуса поворота и столкновений не реализована. Отказы оборудования, случайные времена задач и сезонный график пока не моделируются. Нормы пола, связи, шума и зарядного электроснабжения требуют подтверждения при обследовании.

Экономика

Согласованная модель сравнивает текущий процесс, покупку и аренду без выкупа. По умолчанию горизонт пять лет, доступно изменение до пятнадцати. Собственное финансирование, постоянные цены с НДС. Налоговые эффекты, ликвидационная стоимость и долг не рассчитываются. Дополнительные доходы не предполагаются.

CAPEX покупки: роботы с резервом, зарядки по группам, инфраструктура, ПО, интеграция, обучение и резерв бюджета. OPEX: оставшийся персонал, ручные этапы, прочие сохраняемые расходы, энергия, сервис, лицензии. Для аренды каталог должен явно включать роботов, зарядки, ПО и сервис; энергия и персонал добавляются отдельно. Разовый запуск аренды задаётся пользователем, тариф действует весь горизонт. Компонентные замены отдельно не начисляются в этой версии, их стоимость должна покрываться сервисным нормативом либо сохраняемыми расходами. Это допущение требует проверки для реальной сделки.

$TCO = CAPEX + OPEX \times \text{горизонт}$. Годовой денежный эффект = OPEX текущего процесса минус OPEX варианта. Простая окупаемость = $CAPEX / \text{положительный эффект}$. ROI по T3 = $\text{накопленный эффект} / CAPEX \times 100\%$. Чистый ROI дополнительно вычитает CAPEX. NPV дисконтирует годовые эффекты к моменту начала проекта и вычитает начальные затраты. CAPEX учитывается один раз; неденежная амортизация не вычитается повторно из денежного потока. Отдельная бухгалтерская прибыль не заявляется.

Чувствительность меняет стоимость покупки, стоимость труда и объём операций на $\pm 20\%$. Состав парка фиксирован. Изменение потока повторно проверяет цепочку, и невыполнимый вариант не получает экономику. Малый стартовый бюджет помечает превышение и не делает технический вариант несовместимым. Корректировка цены пользователем сохраняется в версии и делает расчёт предварительным.

Архитектура, API и безопасность

Браузер работает с Django по HTTPS. ORM записывает PostgreSQL. Основные сущности: Supplier, Equipment, EquipmentRevision, ImportBatch, ImportRow, Study, StudyVersion, Configuration, Procurement, SupplierRequest, Message, Notice и DemoOrder. Требования и результаты хранятся снимками JSON с версиями. Каталог организатора из старой модели core сохраняется отдельно.

Новые HTML-маршруты используют сессионную авторизацию и CSRF. POST /studies/upload/ принимает CSV параметров, POST /workspace/upload/ принимает CSV каталога, POST /workspace/parser/ принимает JSONL ZIP только от администратора. GET /configurations/{id}/export/ возвращает JSON; format=pdf или xlsx возвращает отчёт владельцу. POST /configurations/{id}/recheck/ создаёт новый расчёт. Маршруты заявок и заказов определены в workflow/urls.py. Существующие JSON API /api/* описаны в openapi.json и относятся к базовому модулю core.

Будущие адаптеры WMS, ERP и 1С должны передавать нормализованные операции, объёмы и стоимость в ту же проверку требований. Адаптер поставщика должен создавать черновик с доказательствами через importer. Мониторинг оборудования может поставлять фактические времена задач для калибровки модели. Реальные интеграции в конкурсной версии не подключены.

Пароли хешируются Django. Доступ ограничен владельцем проекта, членством поставщика и ролью администратора. У поставщика нет доступа к ФОТ заказчика и чужим предложениям. Экспорт принадлежит владельцу проекта. Срок ожидания ответа составляет семь дней, при открытии заявки показывается просрочка. Фоновая проверка создаёт уведомления каждые 15 минут работы сервера. Во время сна бесплатного хостинга обработка продолжится после пробуждения.

Проверка и передача

`python manage.py test core workflow`, `python manage.py check`, `python manage.py makemigrations --check --dry-run`, `python scripts/verify_release.py`. Перед тестами выполнить `collectstatic`. CI повторяет проверки с SQLite и PostgreSQL.

Демонстрация: стандартный склад, автоматический подбор, смешанная конфигурация, три экономических сценария, воспроизведение, PDF и Excel, заявка всем поставщикам, учебные подтверждения и завершение учебного заказа. Отдельно показать блокировку при чрезмерной массе паллеты и загрузку файла с ошибкой. Старый расчёт должен остаться доступным.

Итоговый пакет: ссылка на код и стенд, инструкция запуска, презентация PPTX/PDF, документация DOCX/PDF, закрыто переданные аккаунты, демонстрационные данные и экспортированный пример отчёта. Пароли не включаются в публичную документацию.

Библиотеки: Django (djangoproject.com), SimPy (simpy.readthedocs.io), openpyxl (openpyxl.readthedocs.io), ReportLab (reportlab.com), Gunicorn (gunicorn.org), WhiteNoise (whitenoise.readthedocs.io), psycopg (psycopg.org). Точные версии и остальные зависимости находятся в `requirements.lock`. Проверка реальной нагрузки 50 одновременных пользователей на бесплатном сервере пока не выполнена.